

Sleeping Machines

Computing with races, cancellations and near misses · status report, 25 September 2026

In one page

Sleeping Machines proposes that computation can happen **in time rather than memory**: candidate events race, the first to fire cancels the rest, and the cancelled ones keep a trace of how close they came, which a later teaching signal can use. Every experiment below had its predictions written down before evaluation, and the report includes what did not work.

What holds up

- **A cancelled node can be taught (E4).** Keeping a loser's distance to threshold lets a delayed teacher promote the right answer. At $K = 128$ classes: 0.79 accuracy where the reward-modulated rule is at chance, with 6% of the weight updates of uniform credit.
- **Races decide as fast as the evidence allows (E2).** An accumulator race beats a fixed-time decoder at every decision time and tracks the optimal MSPRT, using only additions and a threshold; 74% of input events are never processed.
- **Learning work tracks activity, not capacity (E5).** With capacity grown 64×, the race's work stays flat, and it needs about **300× fewer weight updates** than a sparse softmax on the same connectivity. In round 2 its accuracy also caught up (0.998 vs 0.998 at 16k classes).
- **Local, event-driven learning reaches about 96% on MNIST (E6, round 3, one seed).** Counterfactual credit 95.9%, fired-only 96.1%, against 89.5% for a frozen random hidden layer and 92.1% for a single racing layer: depth adds about 4 points, hidden learning about 6.

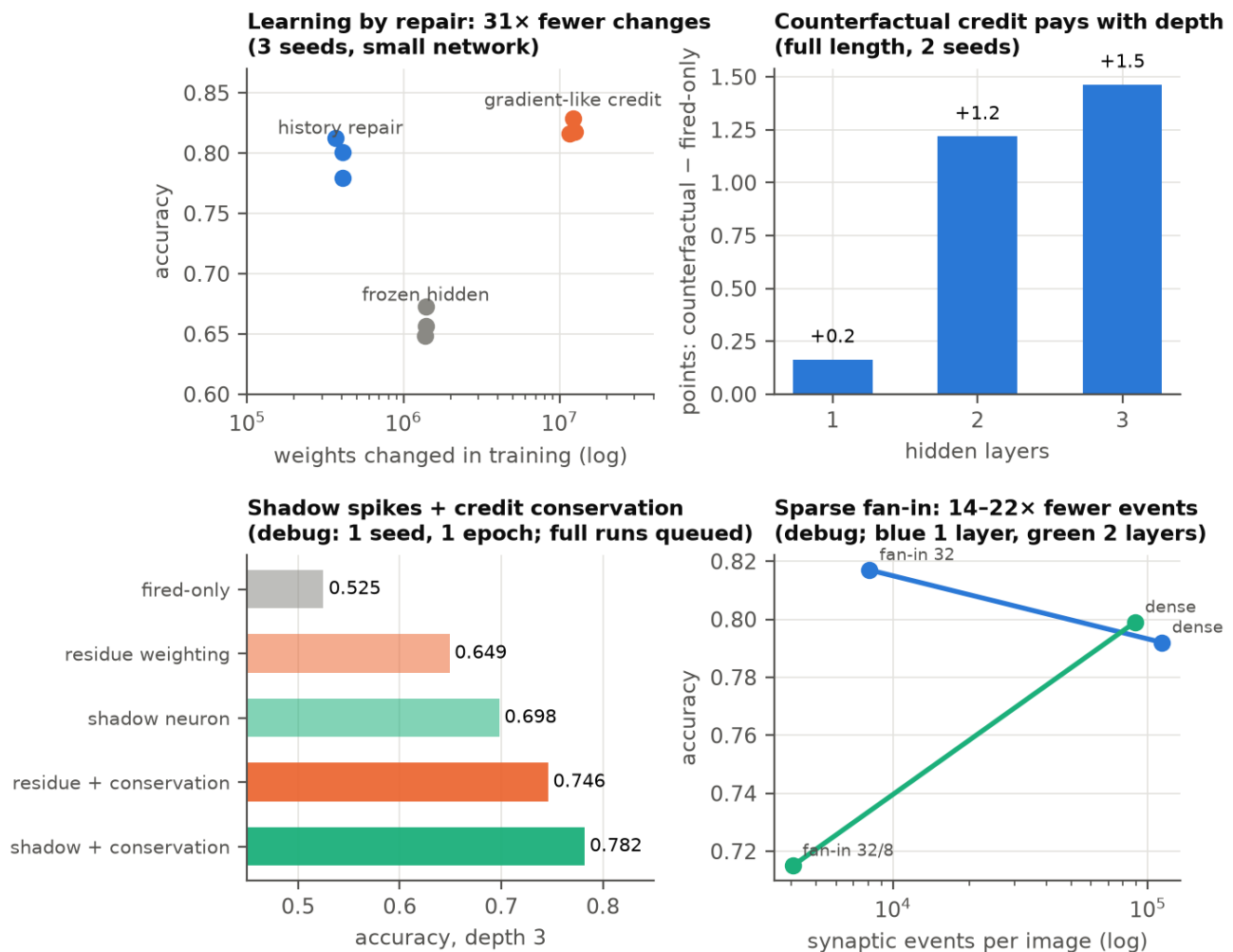
What does not (yet)

- **The counterfactual part of hidden credit adds nothing measurable** over the fired-only ablation (95.9% vs 96.1%). A direct check (M3) finds hidden updates only weakly aligned with the true gradient, although label-driven hidden learning is clearly useful.
- **Stopping hidden work at the decision saves nothing** (0.07%): the cost sits before the decision.
- **Energy.** Only the single racing layer is cheaper than an equally accurate dense model (about 2.4× at inference). The hidden-layer networks use about 108k synaptic events per image, about 4× the multiply-accumulates of a 32-unit MLP that is as accurate.
- **Statistics.** All E6 numbers are single seeds.

What looks promising (details on the next page)

- Learning cost that scales with *errors and activity* rather than with model size (E4, E5), which is exactly what a learner living in a stream needs.
- **History repair** (learning = fixing the pivotal branch point of the event history) comes within ~2.3 points of gradient-like rules while changing **31× fewer weights** (small network, 3 seeds).
- **Reward-only learning:** crediting the near misses when wrong about doubles what the standard reward-modulated spiking rule reaches in a first small test; full runs queued.
- **A theory of learning over the tree of possible event histories** (greedy backprop, holistic backprop and repair as three readings of one tree), and **E7:** learning from one causal stream.

Most promising so far



- **Learning by repairing history** (top left, 3 seeds): fix each mistake at its pivotal branch point with the smallest change. Within ~2 points of gradient-like rules while changing **31× fewer weights**, which matters for continual learning (less interference) and for hardware (fewer writes).
- **Counterfactual credit pays with depth** (top right, full length, 2 seeds): no gain with one hidden layer, +1.2 points with two and +1.5 with three, positive for both seeds. Near-miss nodes influence the output only through events that did not happen, which fired-only credit cannot see.
- **Shadow spikes and credit conservation** (bottom left, debug): letting losing neurons keep integrating and emit shadow spikes into a separate learning channel, and conserving credit at each race (as the theory's soft-minimum derivative requires), lift depth-3 accuracy from 0.65 to 0.78 in a short run. Full runs are queued.
- **Sparse connectivity** (bottom right, debug): 14-22× fewer synaptic events per image. In one layer it even raised accuracy; with two layers it cost accuracy in a very short run. Full runs are queued. For scale, the MLP that matched round-3 accuracy (0.96) needs ~25k multiply-accumulates; sparse race networks at 4-8k events would win at inference if they hold accuracy.
- **Theory that predicts:** reading the network's unrealised futures as a dequantized tropical computation predicted credit conservation (confirmed in debug) and exact time-shift invariance (confirmed); its asynchronous form (shadow events) reproduces batch computation exactly (M20).

Evidence levels: “3 seeds” and “full length” results are confirmatory-grade within their setting; “debug” results are single short runs, reported as leads.

The idea

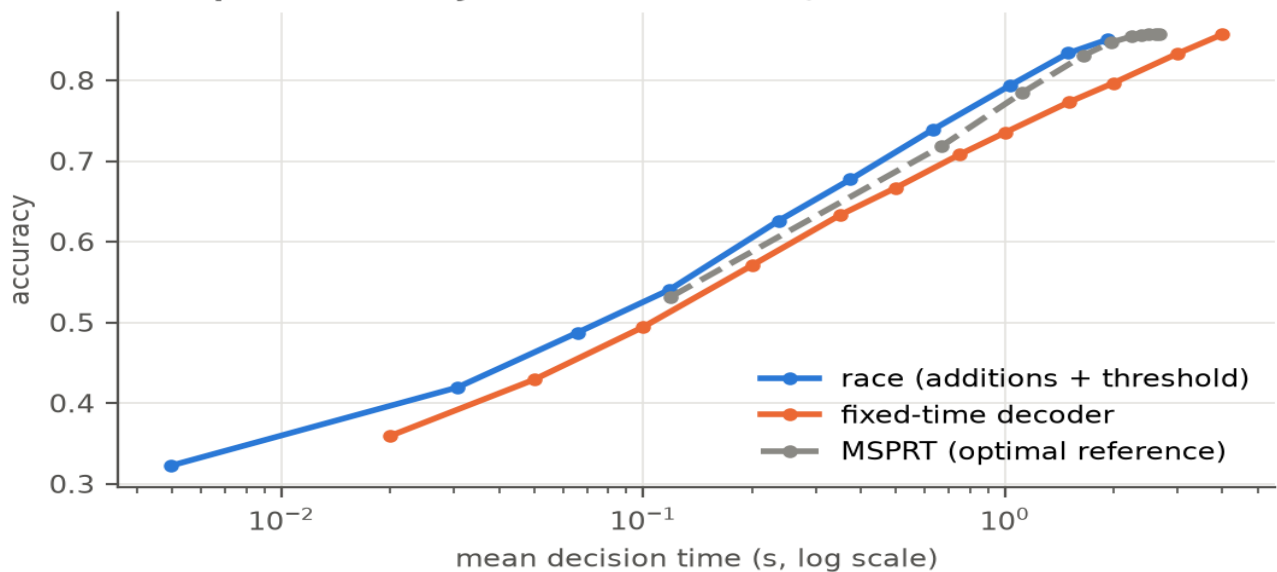
One race: B fires, A and C are cancelled but keep their distance to threshold



Nodes are non-leaky integrate-to-threshold units. The first to reach threshold fires and cancels the others; each cancelled node freezes Δ , its normalised distance to threshold. A teaching event arriving later can then credit near misses, not only the node that fired. The simulator has no global clock: it jumps from event to event and counts every operation, and all work and energy figures come from those counts.

E2 · decisions that take as long as they need

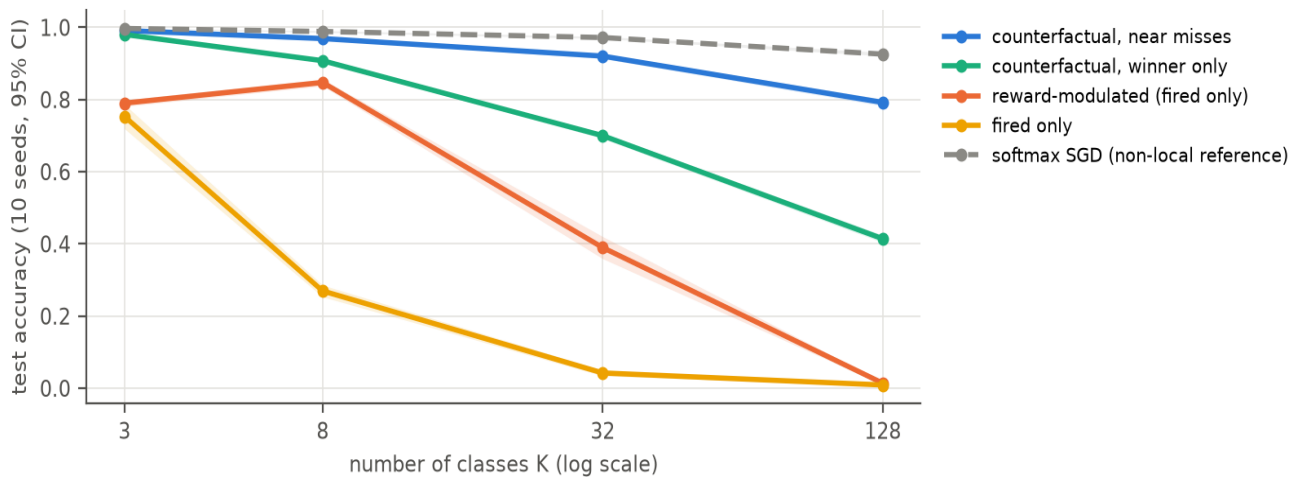
E2 · speed-accuracy frontier (4 classes, Poisson evidence)



The race dominates a fixed-time decoder at every matched decision time and follows the MSPRT, which knows the exact likelihoods. Easy trials end in 0.29 s, hard ones in 1.69 s, with no controller deciding when to stop.

E4 · counterfactual credit

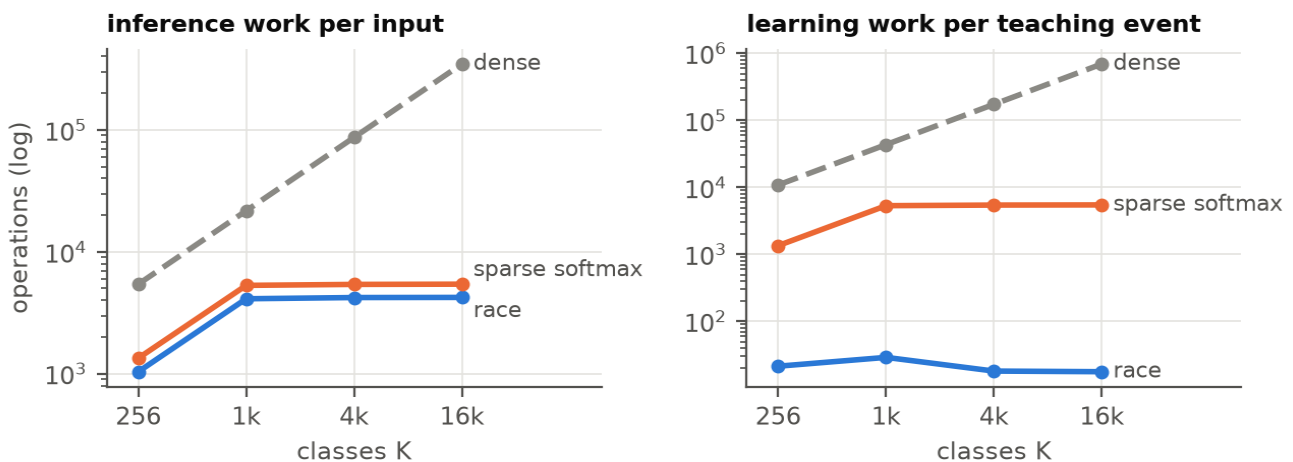
E4 · a cancelled node can be taught



Rules that credit only nodes that fired can only punish a wrong winner; as K grows the right answer almost never fires, and they collapse to chance. Counterfactual rules can promote the target from its frozen record. Crediting only competitors that came close is both the most accurate and the cheapest local rule. It still trails the global-gradient reference by 13 points at K = 128, and credit survives a delayed teacher only until another race overwrites the record.

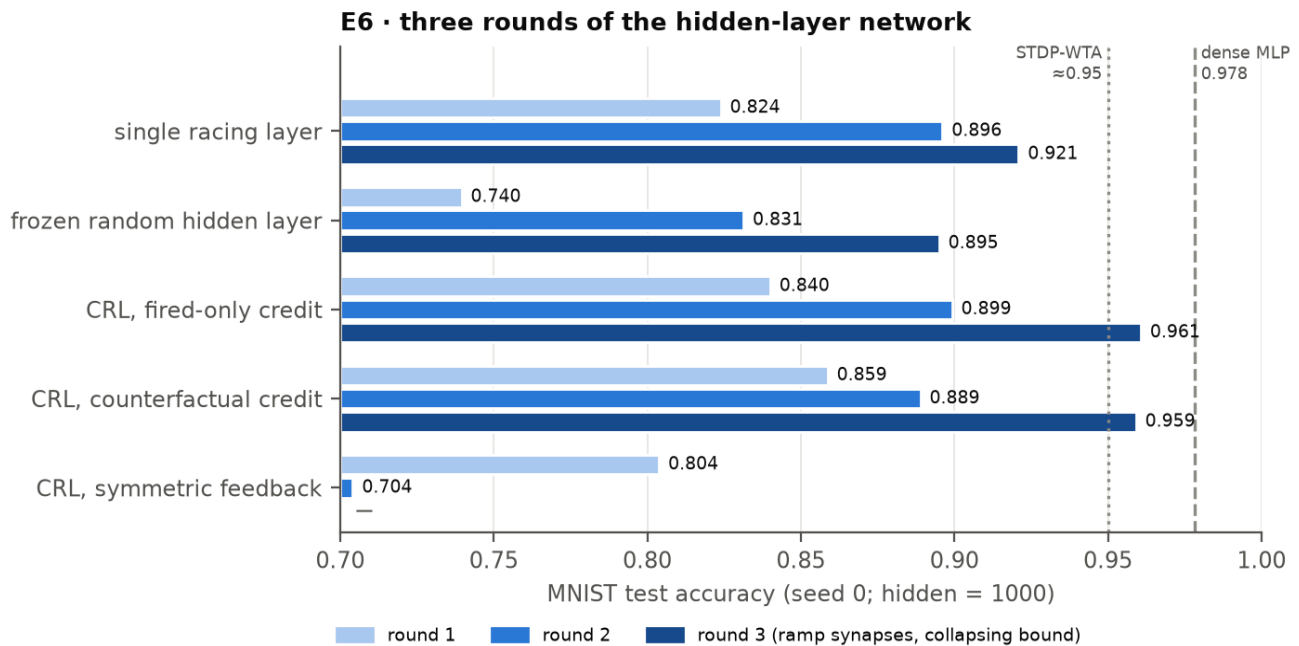
E5 · capacity that costs nothing while it sleeps

E5 round 2 · work as capacity grows 64× (3 seeds)



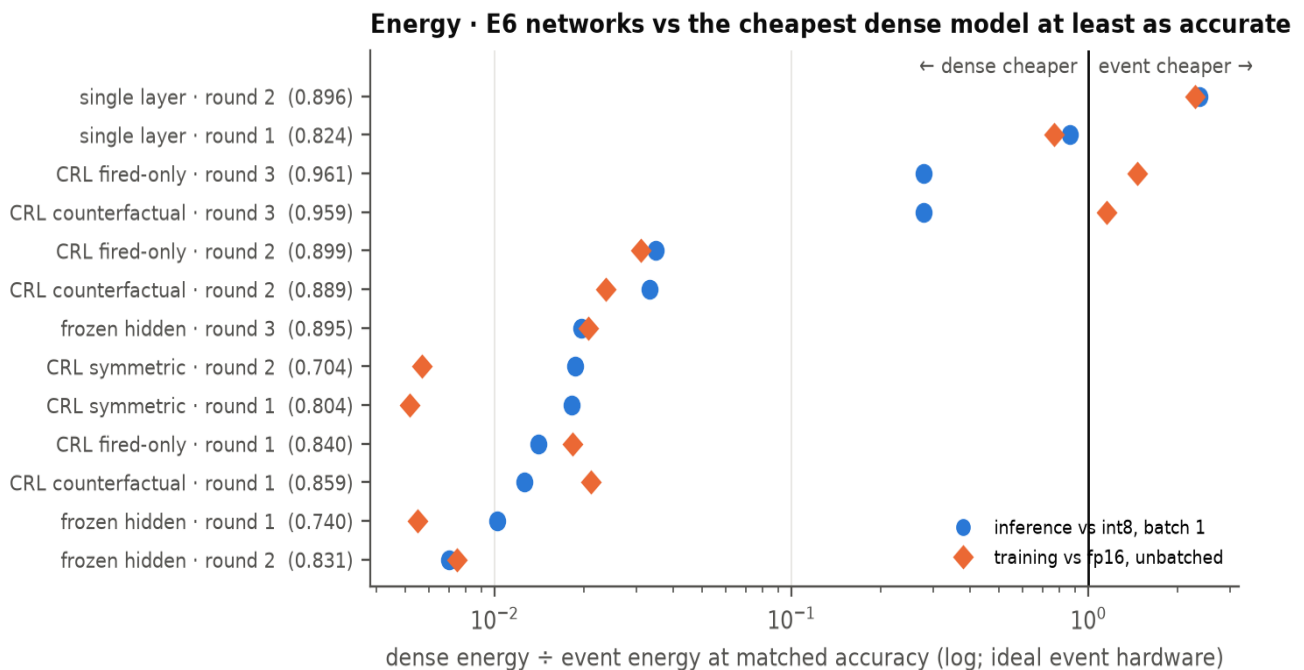
The fair baseline is a sparse softmax on the same connectivity, because most of what event computing saves comes from sparsity itself. Against it, inference work is similar (the race saves about 20% by stopping early), while learning work is about 300× lower: only near misses are updated. Dense models grow linearly with K.

E6 · hidden layers on MNIST



- Round 3 (current-based ramp synapses, a collapsing decision bound, 3 winners per group) lifted the hidden networks from about 0.89–0.90 to about 0.96.
- Hidden learning matters: the frozen random hidden layer stays at 0.895.
- Which hidden credit does not matter yet: fired-only $\approx$ counterfactual. Symmetric feedback was worst in rounds 1–2 and was not carried forward.
- A wider hidden layer (2000 nodes): 96.0%, no gain. The single-layer control reaches 92.1%: the new synapse model gave ~ 2.5 points, depth ~ 4 more.

Energy, measured by counting



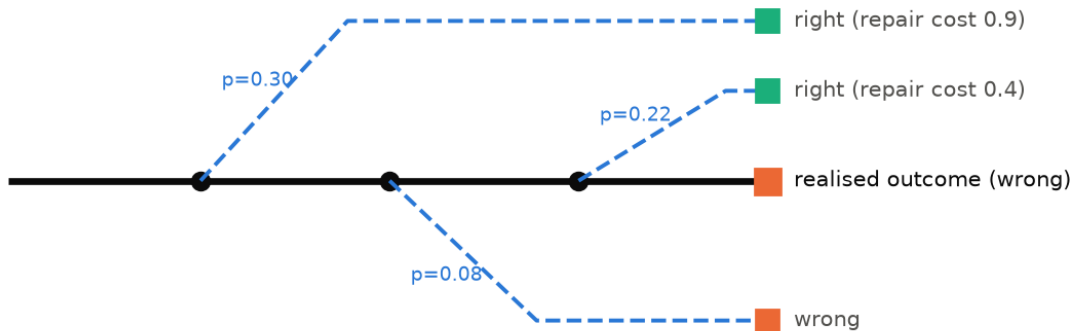
Operation counts priced with published per-operation energies (45 nm logic and SRAM; measured Loihi). These are order-of-magnitude estimates, not chip measurements. At inference, the single racing layer is the only network that beats an equally accurate dense model; the round-3 hidden networks are 1.2–1.5 $\times$ cheaper to *train* than unbatched dense training. Hidden layers integrate about 100 input events in each of 1000 nodes before inhibition stops them, and that swamps the savings. Against

batched dense hardware every advantage disappears.

Theory: collapsing futures, trees of histories, repair

At any moment the network holds a pool of pending futures: each unfired node's projected firing time. A firing collapses the pool: one future becomes history and inhibition cancels its competitors, each leaving a residue (how close it came). Because the order of events depends on the weights, the network's computation is a **tree of possible histories**, and a small weight change can move it to another branch. In a dense network the tree has one branch, so the derivative along it is the whole story. Here it is not.

collapses along the realised history (solid); close calls fork (dashed)

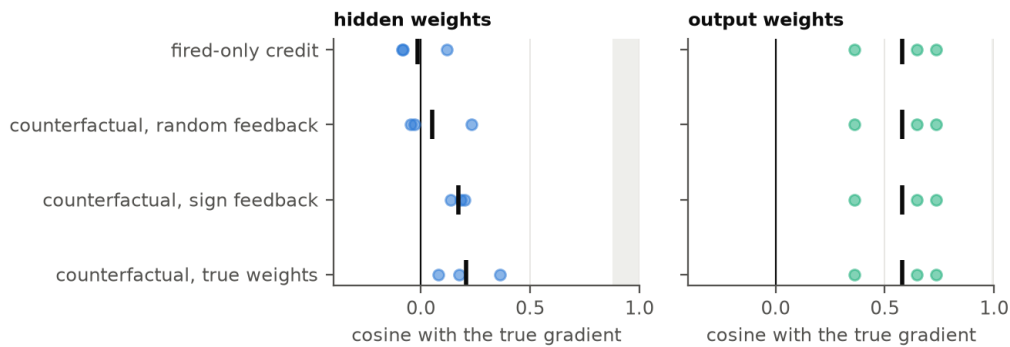


greedy backprop: slope at the solid path only · holistic: sum over forks weighted by p · repair: cheapest fork that is right

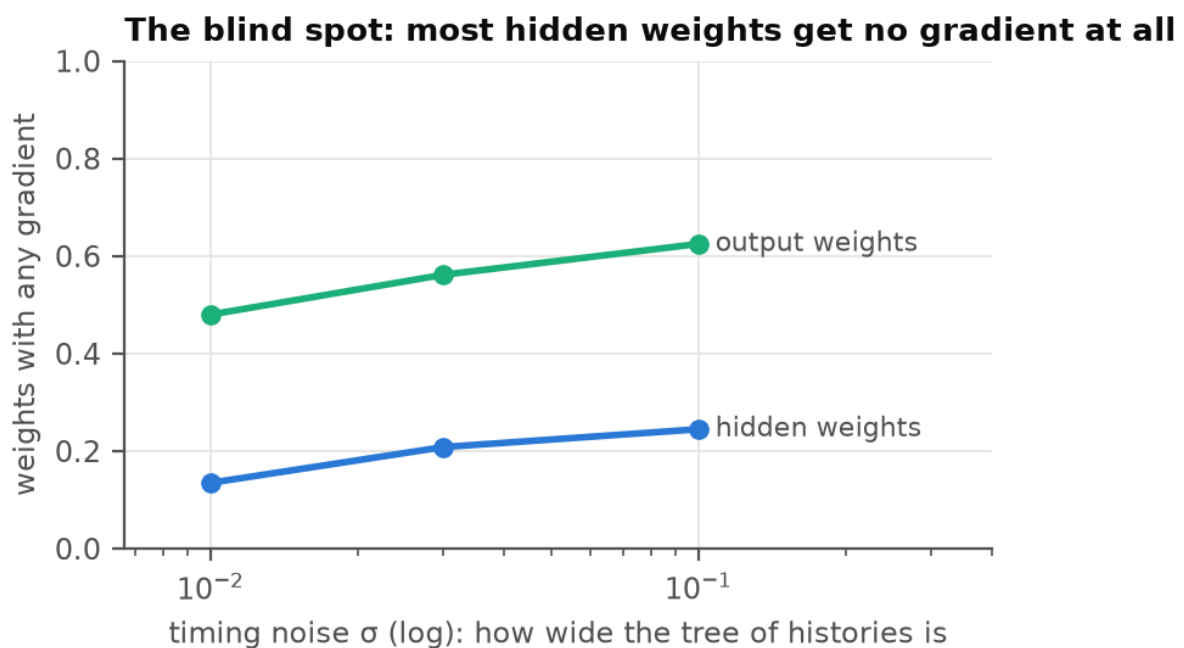
- **Greedy backprop** (EventProp in our setting) follows the realised branch only, and misses how the weights move the forks.
- **Holistic backprop** sums over the close calls, weighted by their probability. Branches are cheap here: a fork changes only events downstream of it, and cancellation ends it early. They can run asynchronously as *shadow events* in the same event queue.
- **History repair** takes the cheapest branch that gives the right answer and changes just that tie. Credit becomes a causal question (would the outcome differ but for this event?) rather than a sensitivity.
- Known ingredients (EventProp, perturbed argmax, surrogate gradients, race logic, Madaline Rule II, spike discontinuity estimation) are credited in experiments/RELATED_WORK.md; the tree-of-histories reading and cancellation residues as its sufficient statistic were not found in prior work.

Checking the theory

M3 · does the update point downhill? (3 seeds; bar = mean; shaded = beyond the estimate's reliability)



M3 measures, on a small network, whether each rule's update points along the true gradient of expected error (estimated by finite differences under timing noise). The output rule does; hidden credit barely does, for every feedback type.

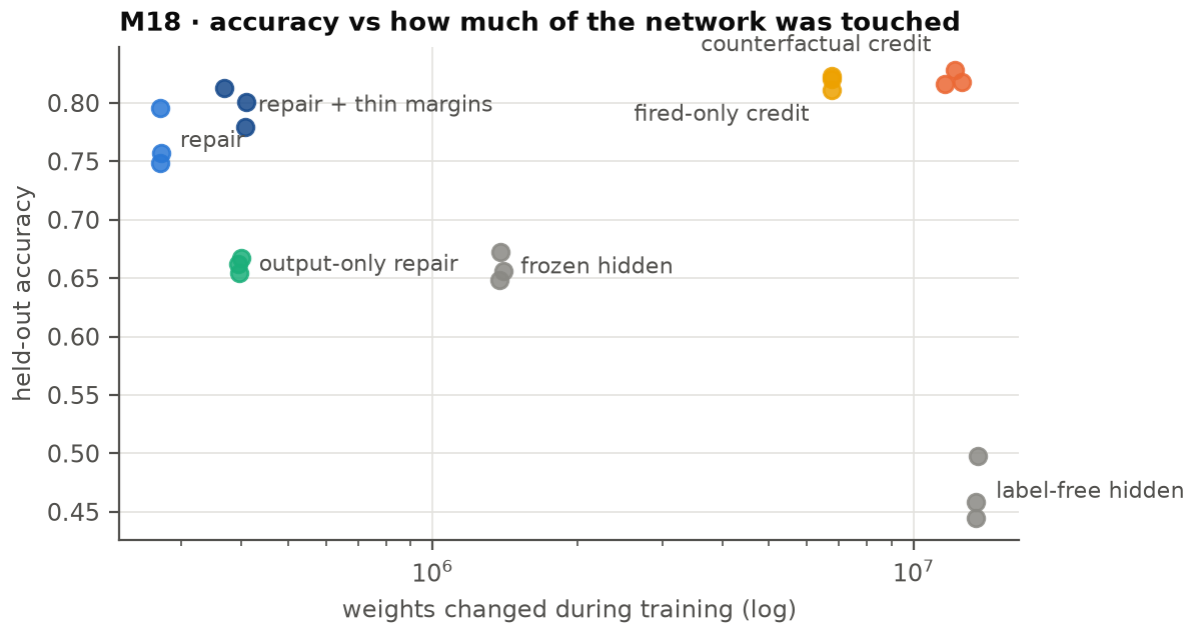


Most hidden weights have no gradient at all along the realised history. Widening the tree (more timing noise) enlarges the set that gets any signal: the blind spot that greedy backprop cannot see.

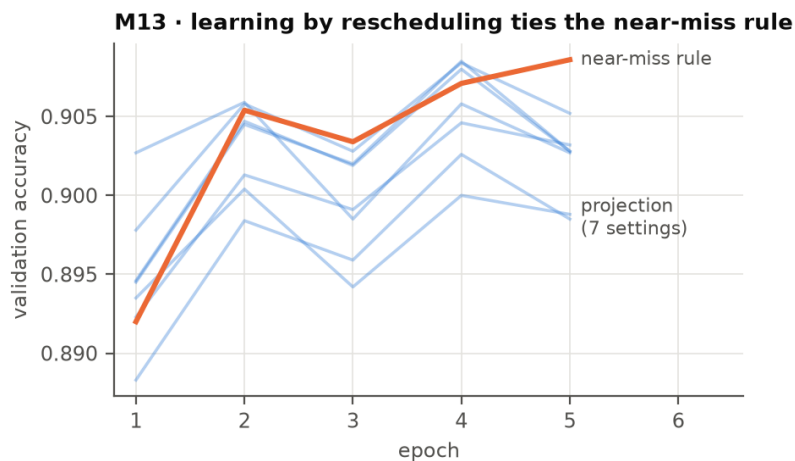
A correction

From M3 we first concluded that the hidden layer learns mainly by its own competition, not from labels. M21 contradicted this: a label-free competitive hidden layer ends far below a frozen random one (0.47 vs 0.66), while label-driven hidden credit adds 16 points. Labels matter to the hidden layer even though their alignment with the gradient of expected error is weak, an open puzzle we are now testing.

New learning rules



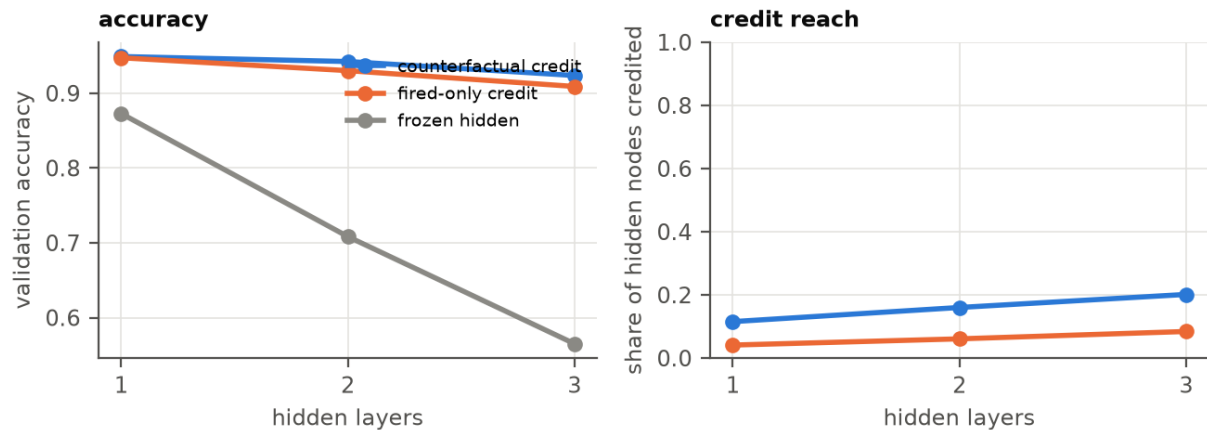
History repair on a small network (14×14 MNIST, 60 hidden nodes, 3 seeds): each mistake is fixed by the cheapest verified single-event change, either rescheduling the output or one hidden node. With thin-margin repairs it reaches 0.80 against 0.82 for the gradient-like rules, while changing 31× fewer weights. Label-free hidden learning (grey, bottom right) is the worst of all.



Learning by rescheduling: “fires by time c ” is a half-space in the weights, so a mistake can be fixed by an exact projection with no learning rate. It ties the near-miss rule: a valid reformulation, not an improvement, and it is now the geometric core of history repair.

Depth: where counterfactual credit pays (E14)

E14 · the advantage of counterfactual credit grows with depth (2 seeds)



Hidden credit per layer through fixed random feedback. With one hidden layer, counterfactual and fired-only credit tie. Deeper, a near-miss node matters only through nodes its spike would have tipped over threshold: a path made of events that did not happen, which only counterfactual credit reaches. At full training the gap is modest (about 2 points at depth 3) but grows steadily with depth, while a frozen hidden stack collapses.

Reinforcement learning: reward only (E13a)

In a first small test, crediting the near misses when wrong reached 0.30, against 0.19 for reward-modulated (winner only) learning and 0.59 with labels. Full runs are queued.

E7 · a learner that lives in a causal stream

Everything above trained the way clocked hardware likes: minibatches, IID epochs, a global learning-rate schedule, a label for every sample, a separate test phase. None of that is needed by an event-driven learner; it is inherited. E7 drops it. The guiding rule is not biological fidelity but this: **where we deviate from biology only because of synchronous-hardware habits, we do not carry the deviation over.**

frames (views of one image) → dark = label arrives orange bar = episode reset



slow state carried across frames: prior · tags · carried label · continuity · homeostasis · consolidation

fast state (one race per frame) restarts at every frame, so the exact solver still applies

each frame is predicted before it may be learned from (prequential); no epochs, no batches

habit	origin	in E7
minibatch updates	GPU throughput	one update per frame (Stage 0 checks this changes nothing)
epochs, reshuffling	batched SGD	one pass over a stream
global learning-rate decay	SGD convergence	per-synapse consolidation
homeostasis from batch means	vectorised statistic	per node, every frame, label or not
dense feedback matrix	cheap matmul	counted as events to eligible nodes only
train phase, then test	benchmark protocol	predict every frame before learning from it
a label for every sample	benchmark convention	10% labels, late labels, or labels the learner asks for
frames with a global $t = 0$	datasets are frames	kept as episode resets; slow state crosses them
latency-coded dense images	MNIST is a frame	kept as a benchmark, not a sensor model

Hypotheses (preregistered)

- **H0** one frame per update learns as well as minibatches of 32.
- **H1** a carried prior cuts the input events needed to decide on later views, without hurting first views.
- **H2** at 10% labels, carried labels and continuity help on real episodes and not on the **shuffled-time control** (same frames, order destroyed).
- **H3** labels the learner asks for (when its race is uncertain) beat random labels on the same budget.
- **H4** synaptic tags make labels that arrive 2 frames late usable.
- **H5** on class-blocked streams the race forgets less than an MLP trained by SGD; replay is reported alongside and may beat both.

Status. Code, tests and the pilot queue are in place; the pilots run after the E6 jobs finish. Pilots use the validation split to set thresholds, which are written into the preregistration before 5-seed confirmatory runs on the test set.

Lessons learned along the way

- **Compute discipline.** Parallel experiment runs hung the host twice (no swap; the second hard reboot corrupted the filesystem). The logs cannot say which job was responsible. Jobs now run strictly one at a time through a queue with a memory cap and a watchdog that stops everything if free memory falls below 3 GB.
- **Hidden batch assumptions.** E6 ran homeostasis inside the teaching step, which silently assumed every sample is taught. With scarce labels that would have switched homeostasis off. Batch-derived constants (homeostasis rate) also have to be rescaled when moving to one frame per update.
- **A case that batching hides.** A frame where no hidden node fires crashed the per-frame learner; in batches, some other sample always fired.
- **Undercounted training work.** The feedback fan-out and homeostasis updates were not counted. They are now, as events to eligible nodes only.
- **Controls before claims.** Round 3 looked like a win for counterfactual hidden credit until the fired-only ablation matched it. The single-layer and shuffled-time controls exist for the same reason.

Where this could go

- **Make the hidden layer cheap:** stop hidden integration as soon as the output race is decided, and use sparse (patch) connectivity, so hidden work stops dominating the energy budget.
- **Truly asynchronous inputs:** N-MNIST (event-camera recordings of MNIST) and Spiking Heidelberg Digits, which need several spikes per input channel.
- **Self-triggered resets:** treat a fixation or saccade the system itself initiates as the reset, so the frame boundary becomes a mechanism rather than a dataset artefact.
- **Seeds:** 3–5 seeds for every headline number before any claim.

Reproduce: `python report/make_pdf.py` rebuilds this document from `experiments/results` and `report/data.json`.
Preregistrations: `experiments/E*_PREREGISTRATION.md`.